



**AMITY INSTITUTE OF INFORMATION TECHNOLOGY
(AIIT)**

**Artificial Intelligence and Machine Learning
(CSE5050)**

Winter Semester – 2025-2026

House Price Prediction

Project Report

SUBMITTED TO:

Dr. Madhu Bhan
(Associate Professor)
(AIIT)

SUBMITTED BY:

Vansh Radadiya - A8691127725002
Smit Kamani - A8691127725004
(MSc Data Science)

Amity University Bengaluru

NH-207, opposite to the office of Deputy Commissioner, Devanahalli,
Doddaballapura, Bangalore – 562110

INDIA REAL ESTATE MARKET DASHBOARD

Item	Details
Project type	Data Science, Real Estate Analytics, Machine Learning, Streamlit Dashboard
Dataset	india_housing_prices.csv, 250,000 records across 20 states and 42 cities
Application file	app.py
Prepared for	M.Sc Data Science Project Submission
Report date	28 June 2026

Abstract:

This project presents an interactive India Real Estate Market Dashboard that combines data analytics, visualization, and machine learning to help users explore housing data, compare regression models, predict house prices, and receive budget-based property recommendations. The objective was to convert a large, difficult-to-read housing dataset into an accessible decision-support tool for students, analysts, buyers, and sellers. The methodology involved cleaning and transforming 250,000 property records from india_housing_prices.csv, building a multi-page Streamlit application, and training and comparing Linear Regression, Random Forest, and Gradient Boosting models for city-level price prediction. The key outcome is a fully functional dashboard with five modules: Dashboard, Dataset, Model Comparison, Predict, and Budget, each verified through smoke testing and screenshot validation.

Introduction:

Real estate prices in India depend on numerous factors such as location, property type, area, number of bedrooms, furnishing status, age, and nearby amenities. Raw tabular data on these factors is difficult for a non-technical user to interpret directly, which limits its practical usefulness for buyers, sellers, and analysts. This project addresses that gap by building an interactive dashboard that filters, visualizes, and models housing data drawn from 20 states and 42 cities across India. The scope of the project extends beyond simple visualization: it includes supervised machine learning for price prediction and a budgetbased recommendation engine, making it relevant both as an academic demonstration of the data science pipeline and as a practical prototype for real-world property search tools.

An earlier version of this work was limited to Mumbai-only property analysis, which restricted its usefulness to a single metropolitan market. The current iteration was deliberately expanded to a nationwide scope so that the same analytical and modeling pipeline could generalize across cities with very different price ranges, property types, and infrastructure characteristics. This expansion increases the academic value of the project, since it requires the underlying code to be written generically rather than hard-coded for one city, and it allows model performance to be examined and compared across multiple, independently trained city-level datasets rather than a single fixed sample.

Methodology

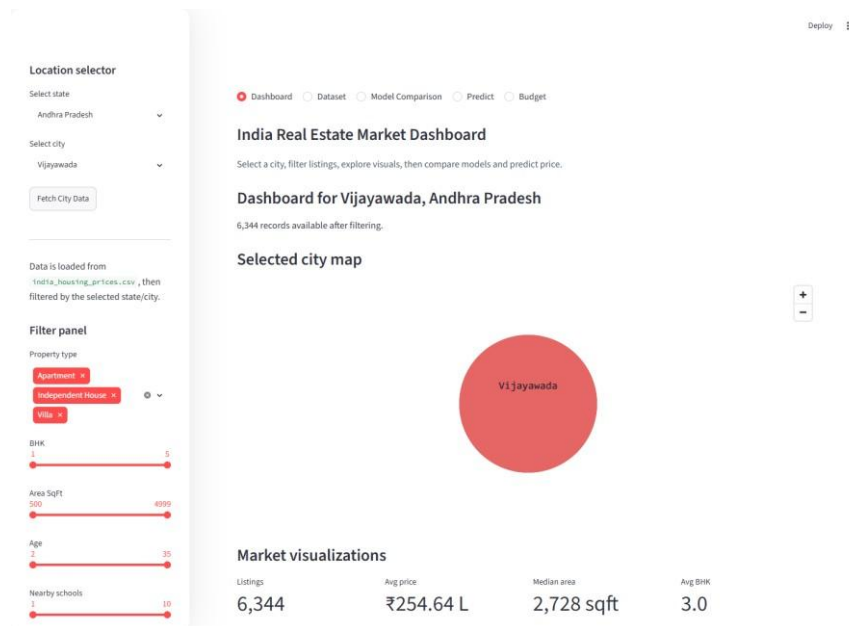
The application is built entirely in Python. Streamlit provides the interactive web interface, sidebar controls, tabs, and forms; pandas and NumPy handle data loading, cleaning, and statistical computation; Altair and PyDeck generate charts and the city map; and scikit-learn provides preprocessing pipelines, regression models, and evaluation metrics. This combination was chosen because it allows a single Python codebase to cover the full pipeline, from raw file ingestion to an interactive, browser-based deployment, without requiring a separate backend service or a JavaScript front end.

The source dataset, `india_housing_prices.csv`, contains 250,000 property records spread across 20 Indian states and 42 cities, with 23 original columns describing attributes such as size, BHK, price per square foot, property type, furnishing, and age. Inside the application, raw column names are mapped to more descriptive, dashboard-friendly names, for example `Size_in_SqFt` becomes `Area SqFt` and `BHK` becomes `Beds`, and the prepared dataset grows to 39 columns once derived fields such as `Amenity Count` are added. Average listed price across the dataset is approximately Rs. 254.59 lakhs, with a minimum of Rs. 10 lakhs and a maximum of Rs. 500 lakhs, spanning property types such as `Villa`, `Independent House`, and `Apartment`.

The stepwise approach followed for the project is summarized below:

- Load the raw `india_housing_prices.csv` file (250,000 rows, 23 columns) and map raw column names to readable dashboard fields such as `Area SqFt`, `Beds`, and `Property Type`.
- Clean text and numeric columns, replace missing or inconsistent values with standardized entries, and engineer a new `Amenity Count` feature from comma-separated amenity lists.
- Allow the user to select a state and city, then apply sidebar filters for property type, BHK, area, age, schools, hospitals, transport, and parking.
- Render dashboard visualizations (map, price distribution, property type comparison, BHK trend, area-versus-price scatter) for the filtered data.
- Split the selected city's data into 80% training and 20% testing sets, then build a `ColumnTransformer` pipeline with median imputation and scaling for numeric fields and mostfrequent imputation with one-hot encoding for categorical fields.
- Train Linear Regression, Random Forest, and Gradient Boosting regressors and evaluate each using RMSE, MAE, RME, RAE, and R2 accuracy to select the best-performing model.
- Use the selected model in a prediction form and in a budget-based recommendation module that ranks properties by a computed `Suggestion Score`.

House Price Prediction Project Report



1. Executive Summary

This project is an interactive India real estate analytics and price prediction system. The older project report focused on Mumbai property analysis, but the current application has been expanded into an India-wide dashboard using the `india_housing_prices.csv` dataset. The system now supports state and city selection, dynamic sidebar filters, map display, market charts, dataset cards, regression model comparison, house price prediction, and budget-based property suggestions.

The application is built in Python using Streamlit for the user interface, pandas and NumPy for data processing, Altair and PyDeck for visualization, and scikit-learn for machine learning. It uses 250,000 property records and transforms raw housing fields into clean analysis columns such as Price in Lakhs, Area SqFt, Beds, Property Type, Furnished Status, Public Transport, Parking Space, Nearby Schools, Nearby Hospitals, and Age.

2. Project Synopsis

Component	Explanation
Project name	India Real Estate Market Dashboard
Main goal	Explore housing data, compare regression models, predict house price, and recommend properties within a user budget.
Target users	Students, analysts, buyers, sellers, and academic evaluators who need an interactive real estate decision-support tool.
Input file	<code>india_housing_prices.csv</code>
Output	Visual dashboard, model comparison table, predicted price, and ranked budget recommendations.

3. Problem Statement and Objectives

Real estate prices depend on many variables including location, property type, BHK, area, furnishing, age, nearby services, transport access, and parking. A raw CSV file is difficult for users to understand directly. This project solves that problem by converting a large real estate dataset into an interactive dashboard and machine learning tool.

- Load and clean India housing data from a CSV file.
- Allow users to select a state and city before analysis.
- Provide filters for property type, BHK, area, property age, nearby schools, nearby hospitals, furnishing, public transport, and parking.
- Show dashboard metrics, map location, market distribution charts, and dataset records.
- Train and compare Linear Regression, Random Forest, and Gradient Boosting models for selected city data.
- Predict house price from user-entered property details.
- Suggest properties that match a user budget and rank them by quality-related factors.

4. Technology Stack

Technology	Purpose in project
Python	Core programming language for data processing, modeling, and dashboard logic.
Streamlit	Creates the interactive web dashboard, sidebar controls, tabs, forms, and data tables.
pandas / NumPy	Loads CSV data, cleans values, maps columns, filters records, and computes statistics.
Altair	Builds price distribution, property type, BHK, and area-vs-price visualizations.
PyDeck	Displays the selected city map using latitude and longitude coordinates.
scikit-learn	Imputation, scaling, one-hot encoding, train-test split, regression models, and evaluation metrics.

5. Dataset Description

The project uses india_housing_prices.csv. The raw dataset contains 250,000 rows and 23 original columns. After mapping and derived fields are added inside the app, the prepared dataset contains 39 columns. The data covers 20 Indian states and 42 city entries.

Dataset statistic	Value
Total records	250,000
Original columns	23

House Price Prediction Project Report

Prepared columns after mapping	39
States	20
Cities / state-city pairs	42
Average price	Rs. 254.59 lakhs
Median price	Rs. 253.87 lakhs
Minimum and maximum price	Rs. 10.00 lakhs to Rs. 500.00 lakhs
Property types	Villa, Independent House, Apartment

The project maps raw column names into dashboard-friendly column names. For example, Size_in_SqFt becomes Area SqFt, BHK becomes Beds, Price_per_SqFt becomes Price per SqFt, Property_Type becomes Property Type, Age_of_Property becomes Age, and Availability_Status becomes Transaction Type.

6. Data Preprocessing and Feature Engineering

The preprocessing workflow is handled mainly by `load_property_data` and `prepare_listing_data` in `app.py`. The first function reads the CSV file, converts important fields into numeric values, and creates user-friendly mapped columns. The second function cleans text values, fills unknown categorical entries, converts numeric columns safely, creates Amenity Count, and removes records without a valid target price.

- CSV loading with UTF-8 encoding and `low_memory` disabled for reliable large-file reading.
- Type conversion for prices, area, BHK, year built, floor number, total floors, age, nearby schools, and nearby hospitals.
- Categorical standardization for property type, furnished status, public transport, parking, security, amenities, facing, owner type, locality, and availability status.
- Missing text cleanup by replacing blank, nan, and None-like values with Unknown.
- Amenity Count feature derived from comma-separated amenities.
- City and state filtering before dashboard rendering and model training.

7. Application Architecture

The project follows a clear Streamlit architecture. Data-loading functions prepare the dataset, sidebar controls filter the data, page-navigation controls choose the active module, visualization functions render dashboard content, and machine-learning functions train and evaluate regression pipelines.

Function / module	Role
<code>load_property_data</code>	Loads CSV data, maps raw columns, and applies optional state/city filters.

House Price Prediction Project Report

prepare_listing_data	Cleans numeric/text columns and prepares listing records for display and modeling.
apply_filter_panel	Creates sidebar filters and returns the filtered dataset.
render_dashboard_page	Shows selected city map and market visualizations.
render_dataset_cards	Shows dataset summary cards and the filtered records table.
train_city_models	Builds preprocessing pipelines, trains three regressors, and returns metrics and best model.
prediction_form	Collects user inputs and predicts house price in lakhs.
render_budget_page	Ranks and displays properties based on budget and suggestion score.

8. Feature Explanation with Project Screenshots

8.1 Dashboard Page

The Dashboard page gives the first analytical view of the selected city. It shows total listings, average price, median area, average BHK, city map, price distribution, property type comparison, BHK trend, and an area-versus-price scatter chart. For the default selected city Vijayawada, Andhra Pradesh, the app shows 6,344 listings, average price of Rs. 254.64 lakhs, median area of 2,728 sqft, and average BHK of 3.0.

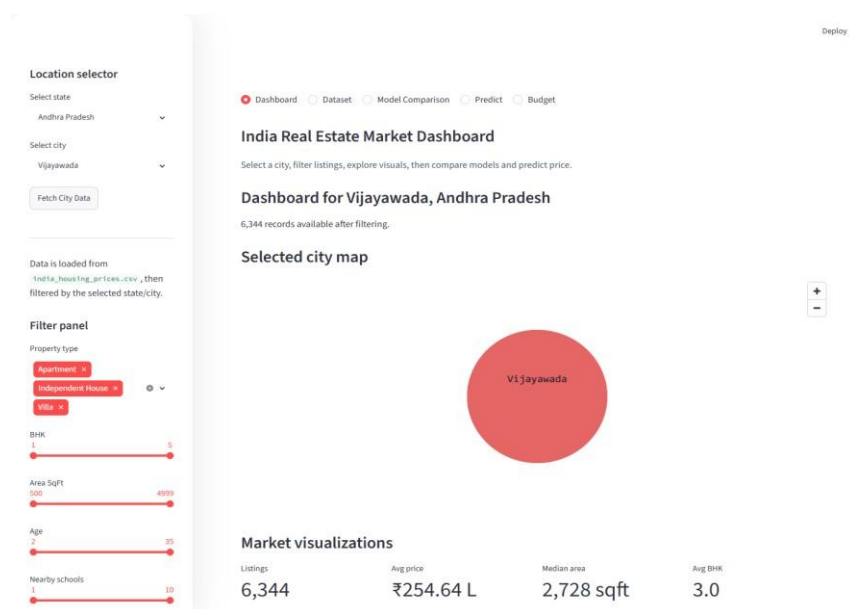


Figure 2: Dashboard page with location selector, filters, city map, and market summary metrics.

8.2 Dataset Page

The Dataset page gives an explainable view of the selected city records. It displays cards for total records, average price, median price, average area, median BHK, average property age, nearby

House Price Prediction Project Report

schools, and nearby hospitals. Below the cards, the user can inspect the complete filtered listing table sorted by price.

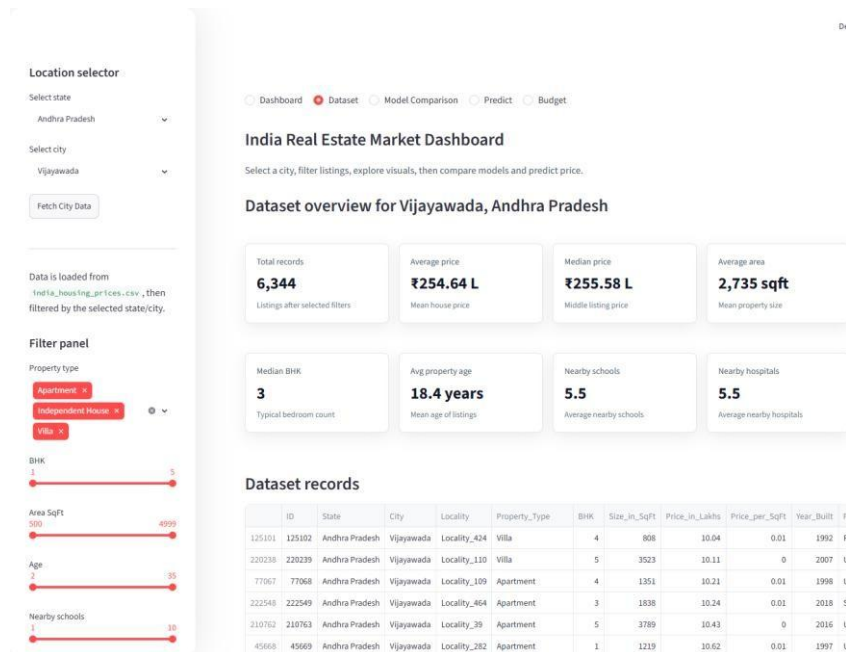


Figure 3: Dataset page with summary cards and detailed listing table.

8.3 Model Comparison Page

The Model Comparison page trains and compares three regression models on the currently selected and filtered city data. The app uses an 80:20 train-test split, preprocessing pipelines, numeric imputation with median values, categorical imputation with most frequent values, onehot encoding, and scaling for numeric features.

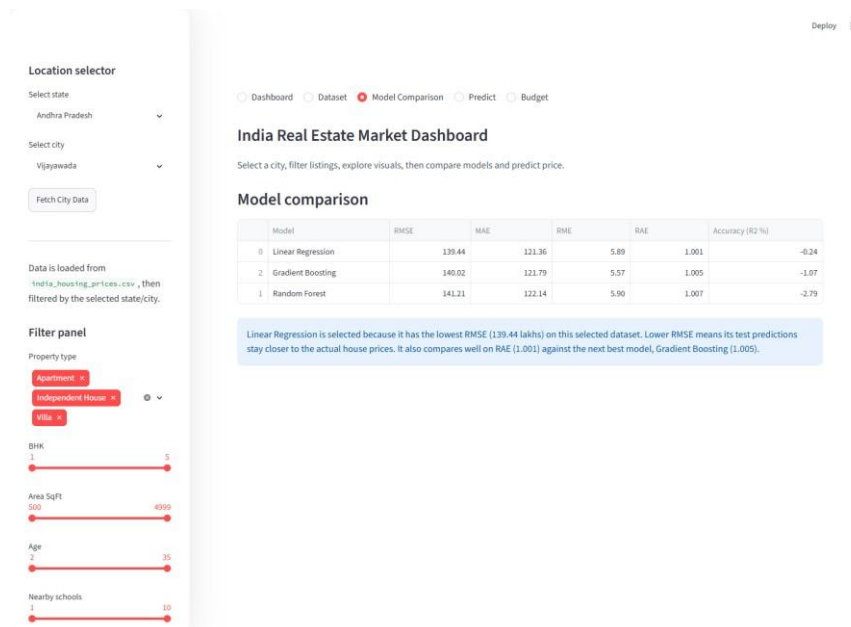


Figure 4: Model comparison table showing RMSE, MAE, RME, RAE, and R2 accuracy.

House Price Prediction Project Report

8.4 Predict Page

The Predict page is the user-facing machine learning form. It trains the best model for the selected city data and then allows the user to enter property details such as property type, beds, area, furnished status, age, nearby schools, nearby hospitals, public transport, and parking space. After submission, the model outputs the predicted house price in lakhs.

Location selector

Select state: Andhra Pradesh

Select city: Vijayawada

Fetch City Data

Data is loaded from 'india_housing_prices.csv', then filtered by the selected state/city.

Filter panel

Property type: Apartment, Independent House, Villa

BHK: 1, 2, 3, 4, 5

Area Sqft: 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 4999

Age: 1, 5, 10, 15, 20, 25, 30, 35

Nearby schools: 1, 5, 10

India Real Estate Market Dashboard

Select a city, filter listings, explore visuals, then compare models and predict price.

Linear Regression is selected because it has the lowest RMSE (139.44 lakhs) on this selected dataset. Lower RMSE means its test predictions stay closer to the actual house prices. It also compares well on RAE (1.001) against the next best model, Gradient Boosting (1.005).

Predict house price

This form uses the best model trained on the currently selected and filtered city dataset.

Property Type: Apartment, Beds: 3.00, Area Sqft: 2728.50

Furnished Status: Furnished, Age: 18.00, Nearby Schools: 5.00

Nearby Hospitals: 5.00, Public Transport: High, Parking Space: No

Predict price

Figure 5: Predict page with dynamic feature inputs generated from the selected city dataset.

8.5 Budget Page

The Budget page helps a user find suitable properties under a chosen maximum budget. It calculates remaining budget for each property and assigns a Suggestion Score using area, BHK, nearby schools, nearby hospitals, and lower property age. The top three matches are shown as cards, followed by a ranked recommendation table.

Location selector

Select state: Andhra Pradesh

Select city: Vijayawada

Fetch City Data

Data is loaded from 'india_housing_prices.csv', then filtered by the selected state/city.

Filter panel

Property type: Apartment, Independent House, Villa

BHK: 1, 2, 3, 4, 5

Area Sqft: 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 4999

Age: 1, 5, 10, 15, 20, 25, 30, 35

Nearby schools: 1, 5, 10

India Real Estate Market Dashboard

Select a city, filter listings, explore visuals, then compare models and predict price.

Budget property suggestions

Set your maximum budget and the best model will suggest matching properties from the selected city data.

Your budget in lakhs: 255.58

Recommendations are ranked for your selected city using budget fit, area, BHK, nearby services, and lower age. The best trained model for this data is Linear Regression.

Villa ₹185.03 L
5 BHK - 4,371 sqft
Remaining: ₹70.55 L
Score: 92.4

Villa ₹169.13 L
5 BHK - 4,049 sqft
Remaining: ₹86.45 L
Score: 91.6

Apartment ₹151.13 L
5 BHK - 4,763 sqft
Remaining: ₹104.45 L
Score: 90.4

	Property Type	Beds	Area Sqft	Furnished Status	Age	Nearby Schools	Nearby Hospitals	Public Transport	Parking Space	Actual Price
60829	Villa	5	4371	Unfurnished	8	10	10	Low	No	
105342	Villa	5	4949	Unfurnished	5	9	7	Medium	No	
90435	Apartment	5	4763	Unfurnished	8	10	7	High	Yes	
232626	Apartment	5	4907	Unfurnished	15	8	10	High	Yes	
33086	Independent House	4	4611	Semi-furnished	4	10	9	Medium	Yes	
13702	Independent House	5	4980	Furnished	7	8	7	High	Yes	

Figure 6: Budget page with recommendation cards and ranked property suggestion table.

9. Machine Learning Methodology

The machine learning target is Price_in_Lakhs. The selected feature columns are Property Type, Beds, Area SqFt, Furnished Status, Age, Nearby Schools, Nearby Hospitals, Public Transport,

House Price Prediction Project Report

and Parking Space. These features are chosen because they are available across the dataset and are understandable to users filling the prediction form.

1. Split the selected city dataset into 80% training data and 20% testing data using `random_state=42`.
2. Use a `ColumnTransformer` to separate numeric and categorical features.
3. For numeric columns, apply median imputation and `StandardScaler`.
4. For categorical columns, apply most-frequent imputation and `OneHotEncoder` with `handle_unknown='ignore'`.
5. Train Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor.
6. Evaluate each model using RMSE, MAE, RME, RAE, and R2 percentage.
7. Select the best model by lowest RMSE and RAE sorting.

10. Model Results and Interpretation

The following metrics were generated for the default selected city dataset: Vijayawada, Andhra Pradesh. In this run, Linear Regression was selected as the best model because it had the lowest RMSE among the three tested models.

Model	RMSE	MAE	RME	RAE	R2 Accuracy
Linear Regression	139.44	121.36	5.89	1.001	-0.24%
Gradient Boosting	140.02	121.79	5.57	1.005	-1.07%
Random Forest	141.21	122.14	5.90	1.007	-2.79%

Important interpretation

The negative R2 values show that the selected features and synthetic-style price distribution do not explain price variation strongly for this city. For academic discussion, this is useful because it shows that model evaluation is not only about building a model, but also about understanding whether available features are predictive enough.

11. Testing and Validation

A smoke test was executed using `smoke_test.py`. The test imported `app.py`, loaded the CSV dataset, prepared the data, and printed the resulting shape and columns. The test confirmed that `load_property_data` returns 250,000 rows and `prepare_listing_data` also keeps 250,000 usable rows after cleaning.

Validation item	Result
Application import	Successful
Dataset load	250,000 rows loaded
Prepared dataset	250,000 rows and 39 prepared columns

Screenshot verification	Dashboard, Dataset, Model Comparison, Predict, and Budget pages captured successfully
Main risk found	Pandas4Warning appears for future pandas string dtype behavior; it does not stop the app.

12. Limitations and Future Scope

- Current city map uses fixed coordinates for known city/state combinations, not exact property-level latitude and longitude.
- Model performance is limited by the available features and the relationship between features and price in the dataset.
- The app trains models at runtime for the selected city, which can take time on large filtered datasets.
- Future improvements can include saved trained models, feature importance charts, property-level maps, authentication, PDF report export, and better hyperparameter tuning.
- The future version can also add SHAP or permutation importance so users can understand why a prediction was produced.

13. Conclusion

The final project is a complete data science dashboard that moves from raw CSV data to visual exploration, model comparison, prediction, and recommendation. It demonstrates important academic skills: data ingestion, preprocessing, feature engineering, visualization, supervised regression, metric-based model selection, and deployment through Streamlit.

Compared with the older Mumbai-specific report, the updated project has a wider India dataset, a richer UI, more filtering controls, a dedicated budget recommendation module, and a clearer machine learning pipeline. This makes the project suitable for demonstration as both an analytics system and a practical decision-support application.

References:

1. Streamlit Documentation, streamlit.io/docs.
2. scikit-learn Documentation, scikit-learn.org.
3. pandas Documentation, pandas.pydata.org.
4. Altair Documentation, altair-viz.github.io.
5. india_housing_prices.csv dataset (Kaggle).